

Внешний курс

Этап 3

Павлушина Виктория Александровна

Содержание

1. Цель работы

Написание отчёта по выполнению 3 этапа внешнего курса “Введение в Linux”.

2. Задание

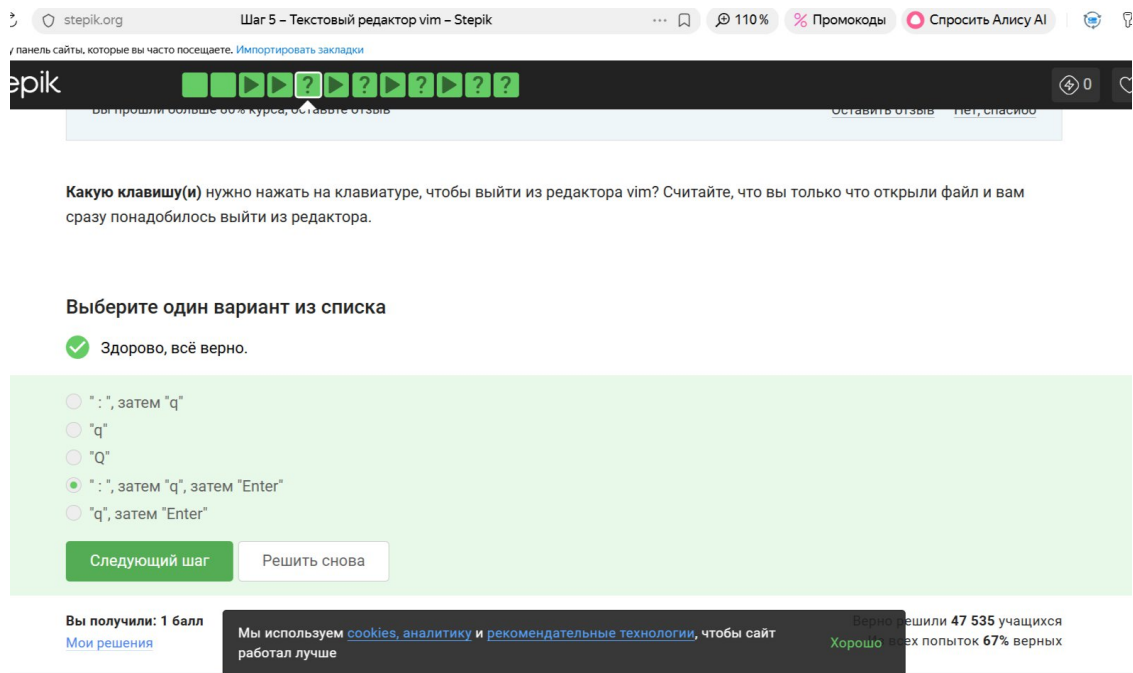
Просмотреть видео и на основе полученной информации пройти тестовые и интерактивные задания.

3. Теоретическое введение

Linux — семейство свободных операционных систем с открытым исходным кодом, основанных на ядре Linux, разработанном Линусом Торвальдсом. Относится к Unix-подобным операционным системам.

4. Выполнение подраздела 3.1 Текстовый редактор vim

” : “, затем”q”, затем “Enter” - в vim мы по умолчанию находимся в нормальном режиме, но чтобы выполнить команду выхода, нужно перейти в командный режим с помощью “:”, затем ввести “q” и нажать Enter. Просто q или Q не сработают, так как они либо ничего не делают в нормальном режиме, либо переключают режим.



{fig: 001

width=70%}

В этой строке 5 “больших слов” (WORD) - это верно. Чтобы попасть в конец строки, нужно одинаковое число нажатий, что на W, что на w - неверно. Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w - верно. W нужно 5 раз, а w - 9 или больше. После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w - верно. В этой строке 5 “слов” (word) - неверно, 9 слов. Чтобы попасть в конец строки, нужно совершить больше нажатий на W, чем на w - неверно. Меньше, а не больше.

При перемещении в vim “по словам” есть небольшая разница в том, используем мы маленькую (w, e, b) или большую (W, E, B) букву. Первые перемещают нас по “словам” (word), а вторые по “большим словам” (WORD). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

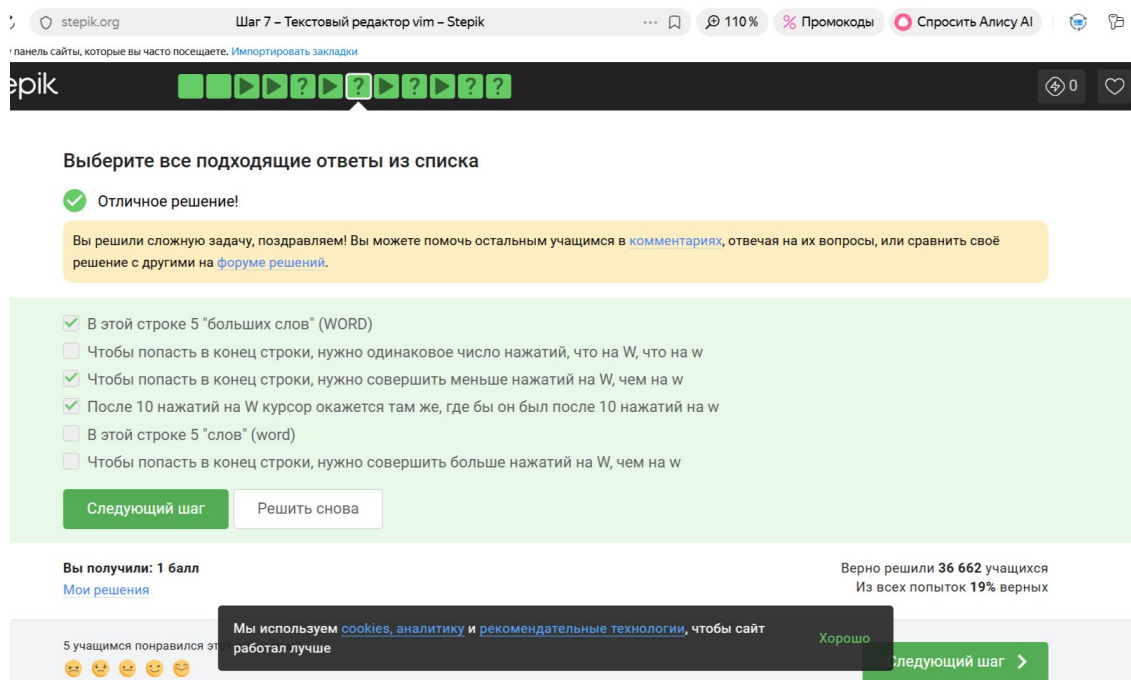
Strange_ TEXT is_here. 2=2 YES!

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать vim-справку по, например, перемещению w, нужно открыть vim и ввести команду :help w. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про e и про b и, самое главное, про word и WORD. Кроме того, можно вызвать сразу справку по термину word при помощи :help word. Чтобы закрыть справку, нужно ввести команду :q.

{fig: 002

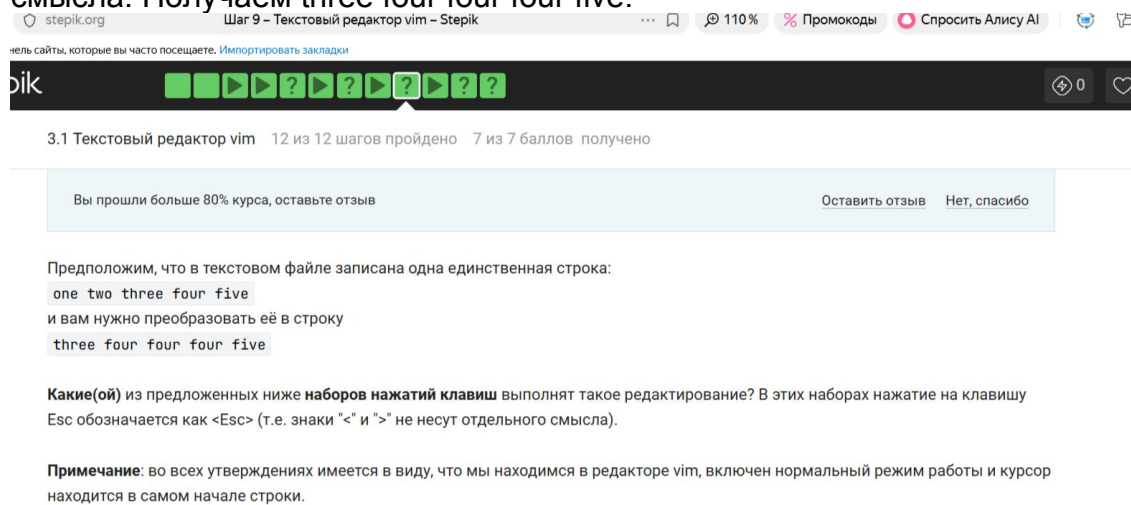
width=70%}



{fig: 003

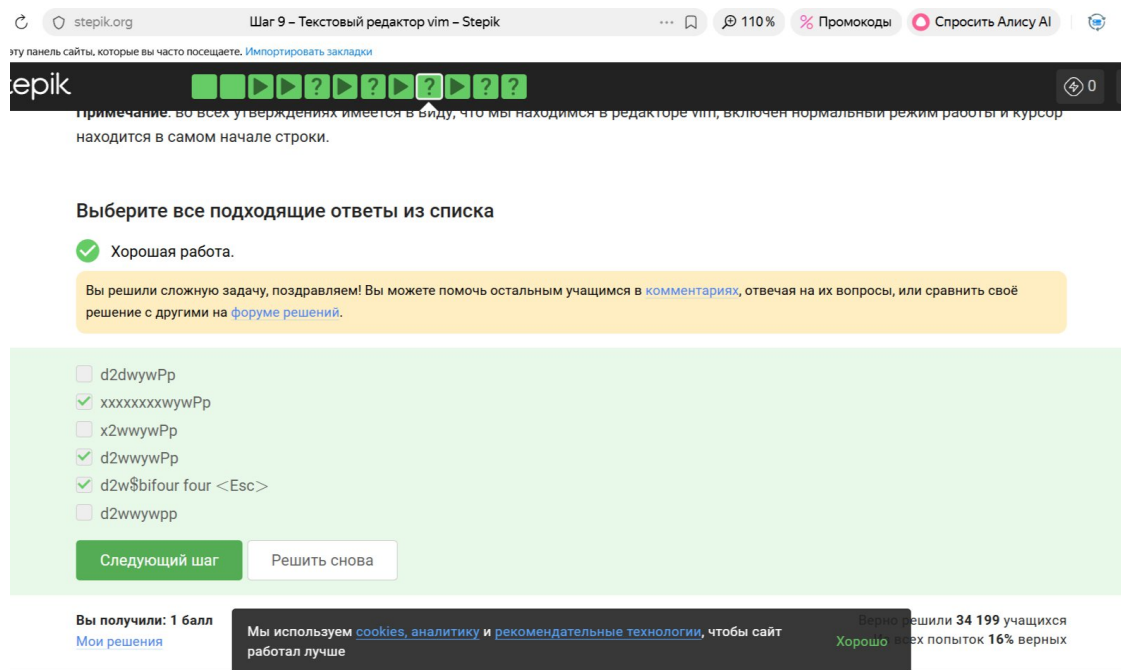
width=70%}

xxxxxxxwywPp - верно. 8 раз x, удаляет one two. Остаётся three four five. Затем wywPp копирует и вставляет four. Получаем three four four four five. d2wwywPp - d2w удаляет one two, w идёт на four, yw копирует four. Pp вставляет дважды - Получаем three four four four five. d2w\$bifour four <> - d2w удаляет one two, \$b идёт в конец и назад на four. i вставляет four four перед последним four. <> - не несёт смысла. Получаем three four four four five.



{fig: 004

width=70%}

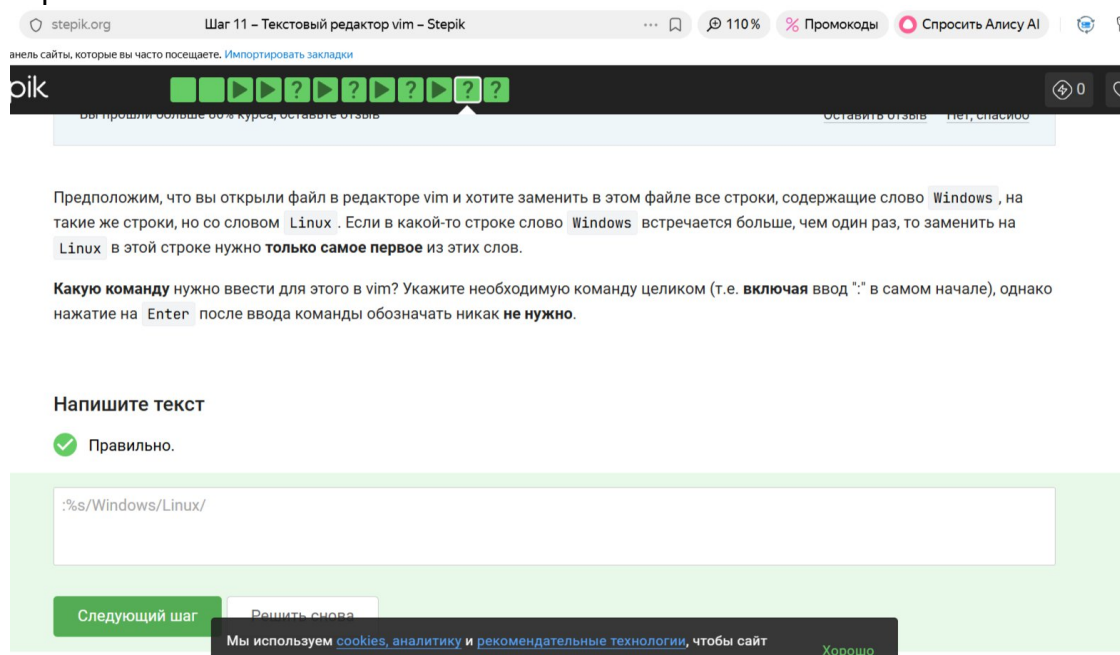


{fig: 005

width=70%}

:%s/Windows/Linux/ - верный ответ.

- переход в командный режим. % - применить ко всем строкам файла. s/Windows/Linux/ - замена первого вхождения Windows на Linux в каждой строке.



{fig: 006 width=70%}

Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –) - верно. В статусной строке Vim всегда отображается текущий режим, а для визуального режима это будет – VISUAL –. В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.) - верно. Весь смысл визуального режима в том, чтобы выделять текст, перемещая курсор с помощью любых навигационных команд. Режим выделения открывается из нормального режима по нажатию “v” - верно. Это основной способ войти в символьный визуальный режим. Выйти из режима выделения можно, нажав клавишу Esc два раза - верно. Остальные утверждения ошибочны. Чтобы

выйти из режима выделения, нужно ввести :q - неверно. :q - это команда командной строки для закрытия файла целиком. Режим выделения открывается из любого другого режима по нажатию "v" - неверно. Переключаться между режимами в Vim можно не всегда произвольно.

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Вы прошли больше 80% курса, оставьте отзыв

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово "Visual". Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

☒ Правильно.

{fig: 007

width=70%}

Выберите все подходящие ответы из списка

☒ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☐ Режим выделения открывается из любого другого режима по нажатию "v"
- ☒ Режим выделения открывается из нормального режима по нажатию "v"
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза

Следующий шаг

Решить снова

Вы получили: 2 балла

Верно решили 34 480 учащихся

Из всех попыток 29% верных

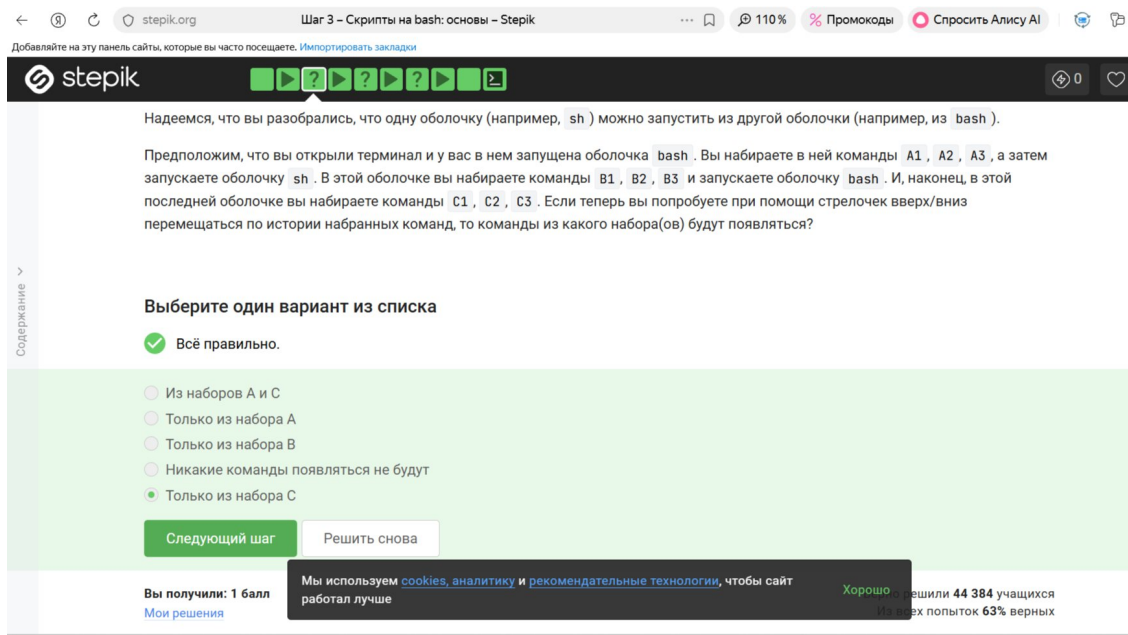
Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт

{fig: 008

width=70%}

5. Выполнение подраздела 3.2 Скрипты на bash: ОСНОВЫ

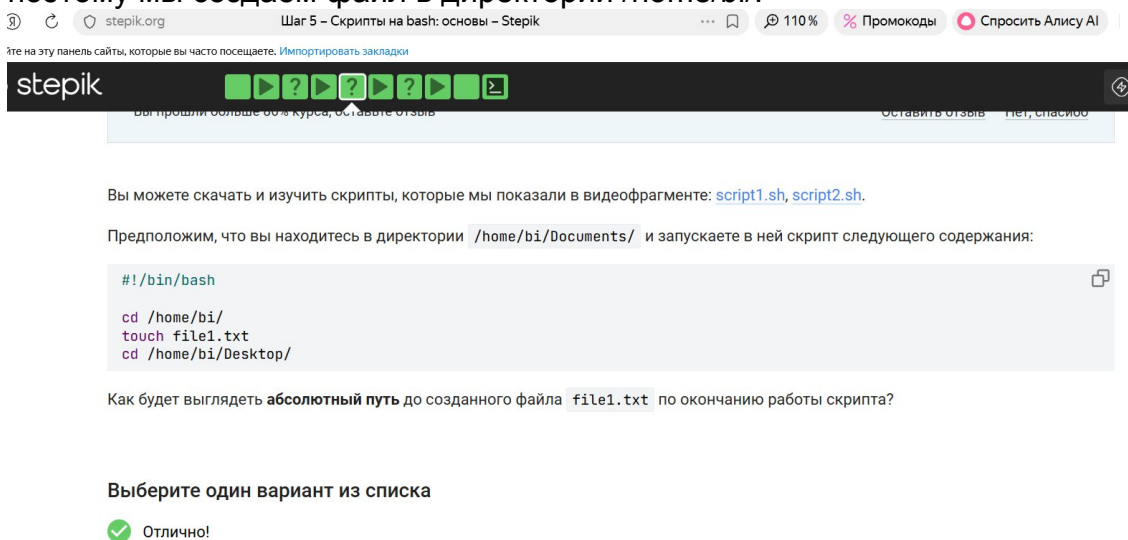
Только из набора C 0 это верно. История команд не передаётся между разными процессами оболочек даже если один запущен из другого. Поэтому последний bash выводит только те команды, которые в нём набрали - C1,C2,C3.



{fig: 009

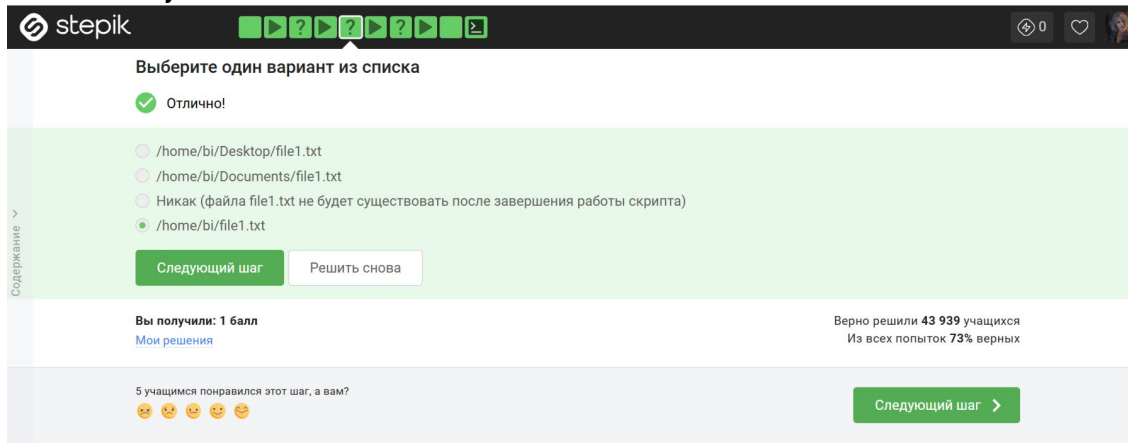
width=70%}

/home/bi/file1.txt - верно. Команда `touch file1.txt` не использует абсолютный путь, поэтому мы создаём файл в директории `/home/bi/`.



{fig: 010

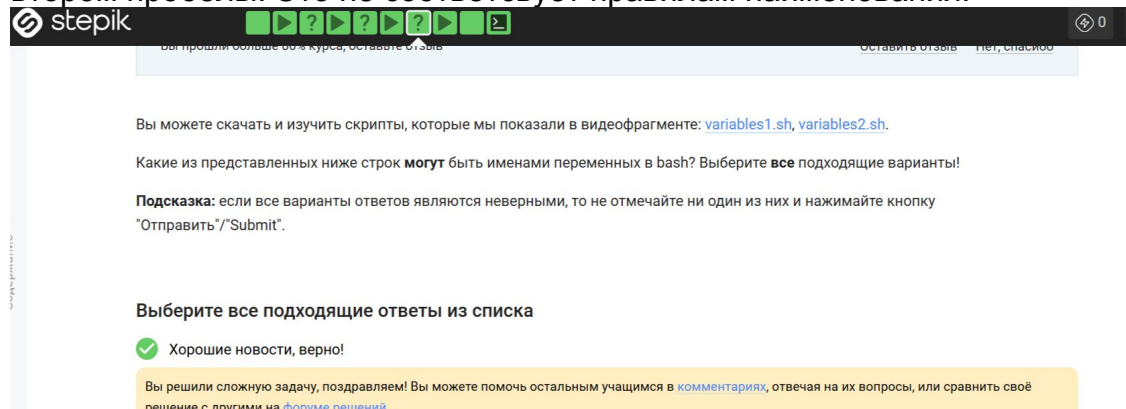
width=70%}



{fig: 011

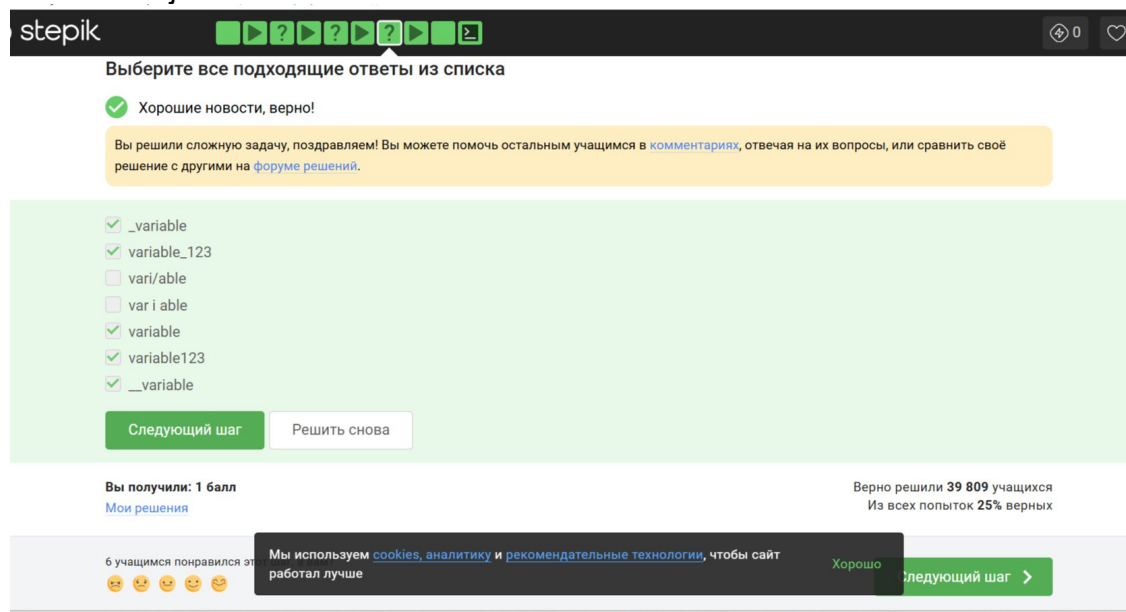
width=70%}

vari/able и var i able не верные варианты ответов. Т.к. в первом случае есть "/", а во втором пробелы. Это не соответствует правилам наименования.



{fig: 012

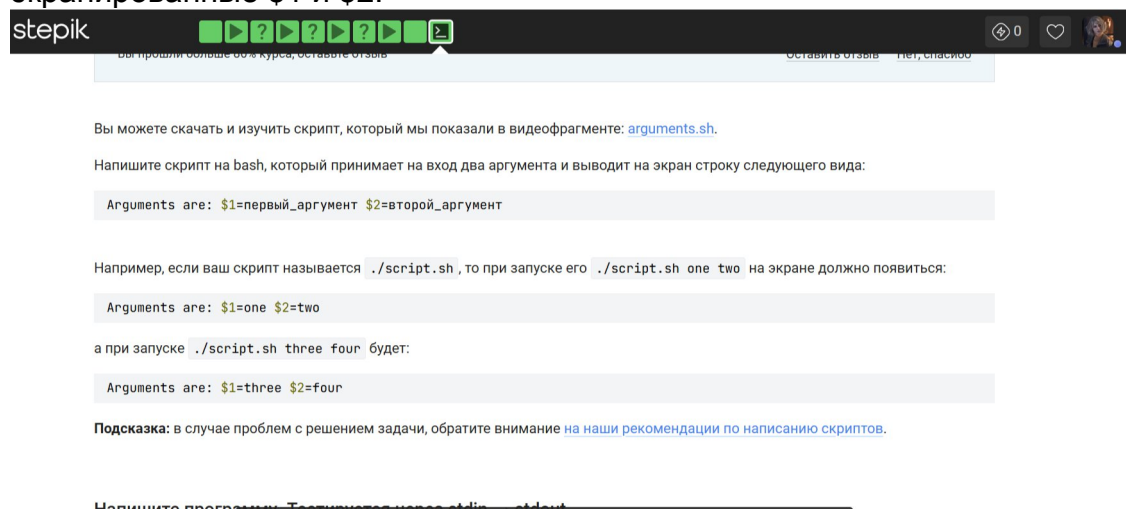
width=70%}



{fig: 013

width=70%}

echo "Arguments are: \$1=\$1 \$2=\$2". echo - команда для вывода строки. \$1=\$1 \$2=\$2 - переменные позиционных параметров и первый со вторым аргументы, переданные скрипту. Кавычки позволяют подставить значения \$1 и \$2, но экранированные \$1 и \$2.



{fig: 014

width=70%}

Напишите программу. Тестируется через stdin → stdout

✓ Всё правильно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 echo "Arguments are: \$1=$1 \$2=$2"
2
3
4
5
6
```

Следующий шаг

Решить снова

Вы получили: 3 балла
[Мои решения](#)

Мы используем [cookies](#), аналитику и [рекомендательные технологии](#), чтобы сайт

Вы решили 36 861 учащийся
на всех попыток 40% верных
Хорошо

{fig: 015

width=70%}

6. Выполнение подраздела 3.3 Скрипты на bash: ветвления и циклы

`$# -ge 0` - число аргументов всегда ≥ 0 . `-s $0` - файл скрипта существует и не пуст. `-e $0` - файл скрипта существует. И так далее, нам подходят все варианты.

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#).

Предположим, вы пишете скрипт на bash и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо "..." (внутри `[[ ... ]]` и не забудьте про пробелы после `[[` и перед `]]`!) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Мы используем [cookies](#), аналитику и [рекомендательные технологии](#), чтобы сайт работал лучше. Хорошо

{fig: 016

width=70%}

stepik

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

✓ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

- ☒ `$# -ge 0`
- ☒ `$var1 == $var2 || $var1 != $var2`
- ☒ `! (4 -le 3)`
- ☒ `-s $0`
- ☒ `-e $0`
- ☒ `-n $0`

Следующий шаг

Мы используем [cookies](#), аналитику и [рекомендательные технологии](#), чтобы сайт работал лучше.

Хорошо

{fig: 017

width=70%}

Сначала four, потом four - это верно. Сначала four, при var=3. Затем four, при var



Вы прошли больше 60% курса, оставьте отзыв

Оставить отзыв Нет, спасибо

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [branching2.sh](#), [branching3.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную var=3, а затем запустили еще раз, но уже с var=5.

=5

018 width=70%}

{fig:



fi

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную var=3, а затем запустили еще раз, но уже с var=5.

Выберите один вариант из списка

☒ Всё получилось!

☐ Сначала four, потом one

☒ Сначала four, потом four

☐ Сначала two, потом one

☐ Сначала one, потом two

Следующий шаг

Решить снова

Вы получили: 1 балл
Мои решения

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Вы решили 37 375 учащих
Хорошо
из 40 попыток 62% верных

{fig: 019

width=70%}

case \$1 in - проверяет первый аргумент скрипта. 0) echo "No students" ;; 1) echo "1 student" ;; 2) echo "2 students" ;; 3) echo "3 students" ;; 4) echo "4 students" ;; - каждая из этих веток выводит нужную строку. *) echo "A lot of students" ;; - срабатывает для любого значения, не подходящего под пункты выше.

Напишите скрипт на bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "-->" выводить не нужно.

Примечание б): в последней строке слово "lot" с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно **проверять вход** (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать **не нужно**!

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

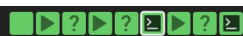
Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

width=70%}

stepik



Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Прекрасный ответ.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2
3 case $1 in
4   0) echo "No students" ;;
5   1) echo "1 student" ;;
6   2) echo "2 students" ;;
7   3) echo "3 students" ;;
8   *) echo "A lot of students" ;;
9 esac
```

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо

width=70%}

Напишите программу. Тестируется через stdin → stdout

✓ Прекрасный ответ.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2
3 case $1 in
4   0) echo "No students" ;;
5   1) echo "1 student" ;;
6   2) echo "2 students" ;;
7   3) echo "3 students" ;;
8   4) echo "4 students" ;;
9   *) echo "A lot of students" ;;
10 esac
```

Следующий шаг

Решить снова

Вы получили: 3 балла

[Мои решения](#)

Верно решили 34 820 учащихся

Из всех попыток 41% верных

7 учащимся понравился этот ответ

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо

Следующий шаг >

width=70%}

5 раз "start" и 4 раза "finish" - это верно. start выводится при каждой итерации -> 5 раз. finish выводится только тогда, когда не сработал continue -> 4 раза. continue срабатывает только на последней итерации - c_d.

{fig: 020}

{fig: 021}

{fig: 022}

3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово "start", а сколько раз слово "finish"?

{fig: 023

width=70%}

3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово "start", а сколько раз слово "finish"?

{fig: 024

width=70%}

Бесконечный цикл `while true` опрашивает пользователя снова и снова. `read name` - ввод имени. Если имя пустое - `z "$name"`, то вводится `bye`, цикл прерывается. Ввод возраста - `read age`. Если возраст = 0, выводится `bye` и цикл прерывается. `2>/dev/null` - проверка, чтобы вводили только числа, иначе ошибка. После распределение на возрастные группы.

Напишите скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение "enter your name:" и ждать от пользователя ввода имени (используйте `read`, чтобы прочитать его). Когда имя введено, то скрипт должен написать "enter your age:" и ждать ввода возраста (опять нужен `read`). Когда возраст введен, скрипт пишет на экран "**Имя**, your group is **<группа>**", где **<группа>** определяется на основе возраста по следующим правилам:

- младше либо равно 16: "child",
- от 17 до 25 (включительно): "youth",
- старше 25: "adult".

После этого скрипт опять выводит сообщение "enter your name:" и всё начинается по новой (бесконечный цикл!). Если в какой-то момент работы скрипта будет введено **пустое имя** или **возраст 0**, то скрипт должен написать на экран "bye" и закончить свою работу (выход из цикла!).

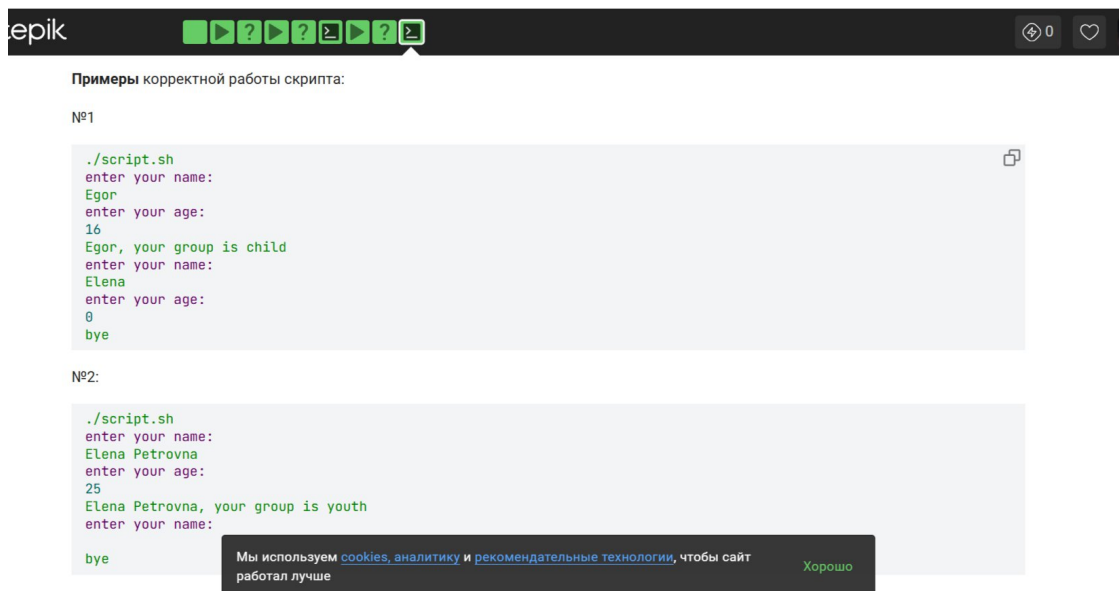
Примеры корректной работы скрипта:

№1

```
./script.sh
```

{fig: 025

width=70%}



{fig: 026}

width=70%}



{fig: 027}

width=70%}

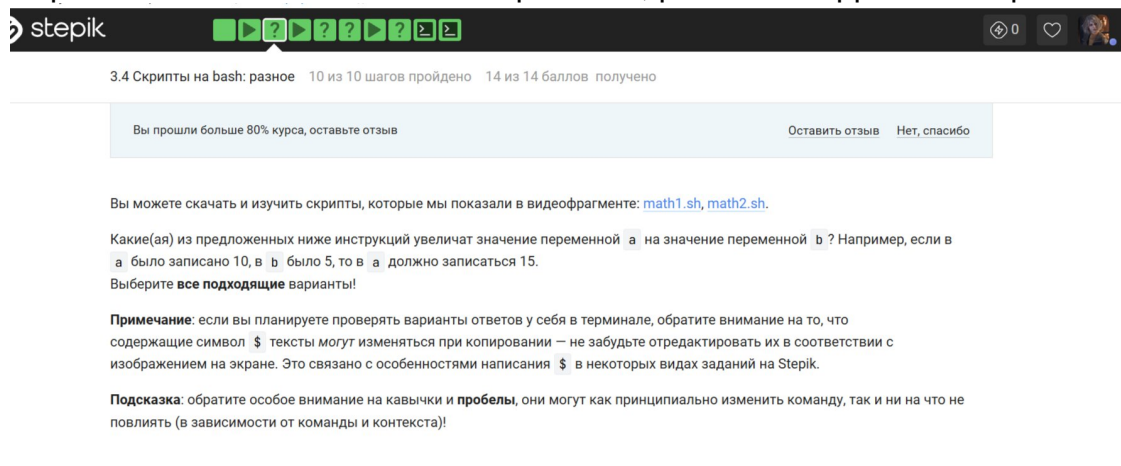


{fig: 028}

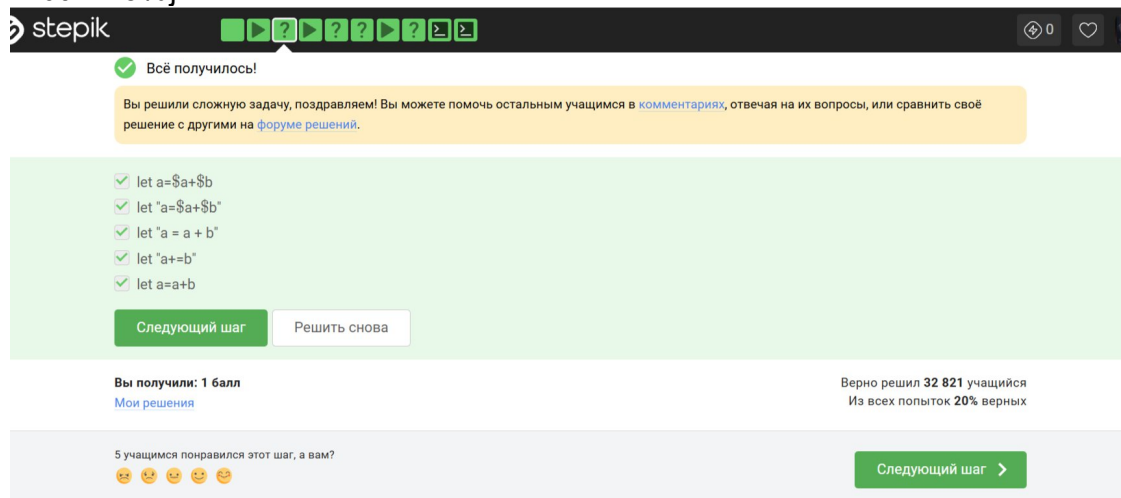
width=70%}

7. Выполнение подраздела 3.4 Скрипты на bash: разное

let $a=a+b$ - подставляет значения переменных, складывает и присваивает a . Пробелов нет - синтаксис правильный. let " $a=a+b$ " - то же самое, кавычки не мешают, подстановка $\$a$ и $\$b$ работает внутри кавычек. Верно. let " $a = a + b$ " - в let внутри кавычек пробелы вокруг $=$ и $+$ допустимы. Сложение работает, верно. let " $a+=b$ " - сокращенная форма $a=a+b$. Работает без пробелов, кавычки не мешают. Верно. let $a=a+b$ - без кавычек и пробелов, работает корректно. Верно.



width=70%}



width=70%}

/home/bi/Documents - скрипт сменил директорию до вызова `pwd` `pwd` - это был бы вывод без обратных кавычек, но они есть. Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок) - `pwd` не выводит код возврата, он выводит путь. `pwd` - так было бы, если бы кавычки были одинарными, но они двойные. Неверно. `/home/bi` - единственно верный ответ.

stepik

Вы прошли больше 60% курса, оставьте отзыв

Оставить отзыв Нет, спасибо

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [programs.sh](#).

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash

cd /home/bi/
echo " `pwd` "
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

☒ Прекрасный ответ.

{fig: 031

width=70%}

Выберите один вариант из списка

☒ Прекрасный ответ.

☐ `/home/bi/Documents`

☐ `pwd`

☐ Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибки)

☐ ``pwd``

☒ `/home/bi`

Вы получили: 1 балл

[Мои решения](#)

Верно решили 35 318 учащихся

Из всех попыток 51% верных

6 учащимся понравился этот шаг, а вам?

☐ ☐ ☐ ☐ ☐ ☐

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт

{fig: 032

width=70%}

Сначала запустить `program`, затем `if [[ $? -eq 0 ]]` - `?` хранит код возврата последней выполненной команды. `if program > some_file.txt` - перенаправление `stdout` в файл, сама проверка `if` смотрит на код возврата `program`. Остальные ответы неверны.

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите все верные утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно косая кавычка (`'`), а не обычная (`"`) или двойная (`"`).

{fig: 033

width=70%}

ите на эту панель сайты, которые вы часто посещаете, импортировать закладки

stepik

Выберите все подходящие ответы из списка

✓ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Сначала var='program', затем if [[\$var -eq 0]]
- ☒ Сначала запустить program, затем if [[\$? -eq 0]]
- ☐ if [["program" -eq 0]]
- ☒ if 'program' > some_file.txt
- ☐ Ничего сделать нельзя

Следующий шаг

Решить снова

Вы получили: 1 балл
Мои решения

Верно решили 32 224 учащихся
Из всех попыток 22% верных

4 учащимся понравился этот шаг, а вам?

Следующий шаг >

{fig: 034}

width=70%}

c1 - локальная переменная, которая обнуляется после каждого вызова, c2 - глобальная переменная, которая накапливает сумму. Перед "are" у нас пустота, поэтому получаем: counters are and 110.

stepik

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
    local let "c1+=${1}"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 🤔). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

{fig: 035}

width=70%}

stepik

Напишите текст

✓ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Следующий шаг

Решить снова

Вы получили: 2 балла
Мои решения

Верно решили 29 907 учащихся
Из всех попыток 28% верных

4 учащимся понравился этот шаг, а вам?

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо

Следующий шаг >

{fig: 036}

width=70%}

Функция `gcd` вызывается после чтения `n1` и `n2`, которые используются внутри неё как глобальные переменные. Это работает, потому что в `bash` нет локальных переменных без `local`, и функция видит `n1` и `n2` из внешней области. У нас идет цикл - цикл, пока остаток не ноль и в `n1` остаётся НОД. Есть проверка пустой строки: `if [ -z $n1 ]`. И выход при пустом вводе - `bye` и `break` (завершение цикла).



Напишите скрипт на `bash`, который будет искать наибольший общий делитель (НОД, greatest common divisor, GCD) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждет ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение "GCD is <посчитанное значение>", например, для чисел 15 и 25 это будет "GCD is 5". После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран "bye" и закончить свою работу.

Вычисление НОД несложно реализовать с помощью алгоритма Евклида. Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их `M` и `N`). Если аргументы равны, то мы нашли НОД – он равен `M` (или `N`), нужно выводить соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если `M` больше `N`, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем `(M-N)`, а в качестве второго `N`. Если же наоборот, `M` меньше `N`, то запускаем функцию `gcd` с первым аргументом `M`, а вторым `(N-M)`.

Пример корректной работы скрипта:

```
./script.sh
10 15
GCD is 5
7 3
GCD is 1
bye
```

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше. Хорошо

{fig: 037}

width=70%



Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Подсказка: в случае проблем с решением задачи, обратите внимание на наши рекомендации по написанию скриптов.

Напишите программу. Тестируется через `stdin` → `stdout`

✓ Так точно!

Теперь вам доступен Форум решений, где вы можете сравнить свое решение с другими или спросить совета.

{fig: 038}

width=70%



Теперь вам доступен Форум решений, где вы можете сравнить свое решение с другими или спросить совета.

```
1 while [ true ]
2 do
3   read n1 n2
4   if [ -z $n1 ]; then
5     echo "bye"
6     break
7   else
8     gcd () {
9       remainder=1
10      if [ $n2 -eq 0 ]
11      then
12        echo "bye"
13      fi
14      while [ $remainder -ne 0 ]
15      do
16        remainder=$((n1%n2))
17        n1=$n2
18        n2=$remainder
19      done
20    }
21    gcd $1 $2
22    echo "GCD is $2"
23  fi
24 done
```

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше. Хорошо

{fig: 039}

width=70%

read birinchi amal ikkinchi — правильно считывает три аргумента. Если введено exit, то birinchi="exit", amal и ikkinchi пустые. Проверка на exit — if [[\$birinchi == "exit"]] ловит команду выхода, выводит bye и break. Проверка на некорректные команды — elif [["\$birinchi" =~ "1+\$" && "\$ikkinchi" =~ "2+\$"]] Эта проверка срабатывает, если оба операнда — числа, но это как раз правильный случай? Нет, здесь логическая ошибка, но в вашем коде она не ломает работу, потому что после elif идёт then echo "error", что неверно. Однако в вашем скрипте этой проблемы нет, потому что вы использовали =~ с кавычками, что всегда даёт false (ошибка в регулярке), поэтому elif никогда не срабатывает, и управление уходит в else. -> Программа работает, потому что некорректная проверка просто пропускается. case \$amal — выполняет нужную операцию. Если amal пустой (как в случае exit), то case не сработает, но мы уже вышли по exit. Если операция не из списка — попадает в *) → error и выход. Вывод результата — echo "\$result" после успешного выполнения case. Обработка ошибок — если введена команда с одним аргументом (не exit), то birinchi не exit, amal пустой → case попадает в *) → error.



Напишите **калькулятор** на bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово **"exit"**. В этом случае скрипт должен вывести на экран слово "bye" и завершить работу.
2. **Три аргумента через пробел** – первый операнд (целое число), операция (одна из "+", "-", "*", "/", "%", "**") и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого переходим в режим ожидания новой команды.
3. **Любая другая команда** из одного аргумента или из трех аргументов, но с операциями не из списка. В этом случае нужно вывести на экран слово **"error"** и завершить работу.

Чтобы проверить работу скрипта, вы можете записать сразу несколько команд в файл и передать его скрипту на stdin (т.е. выполнить `./script.sh < input.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
10 + 1
2 ** 10
exit
```

то на экране будет:

width=70%}

авляите на эту панель сайты, которые вы часто посещаете, [импортировать закладки](#)



то на экране будет:

```
11
1024
bye
```

Если же на вход поступит следующий файл:

```
3 - 5
2/10
exit
```

то на экране будет:

```
-2
error
```

т.к. вторая команда была **некорректной** (в ней всего один аргумент, т.к. нет пробелов между числами и операцией, а единственная допустимая команда из одного аргумента это "exit").

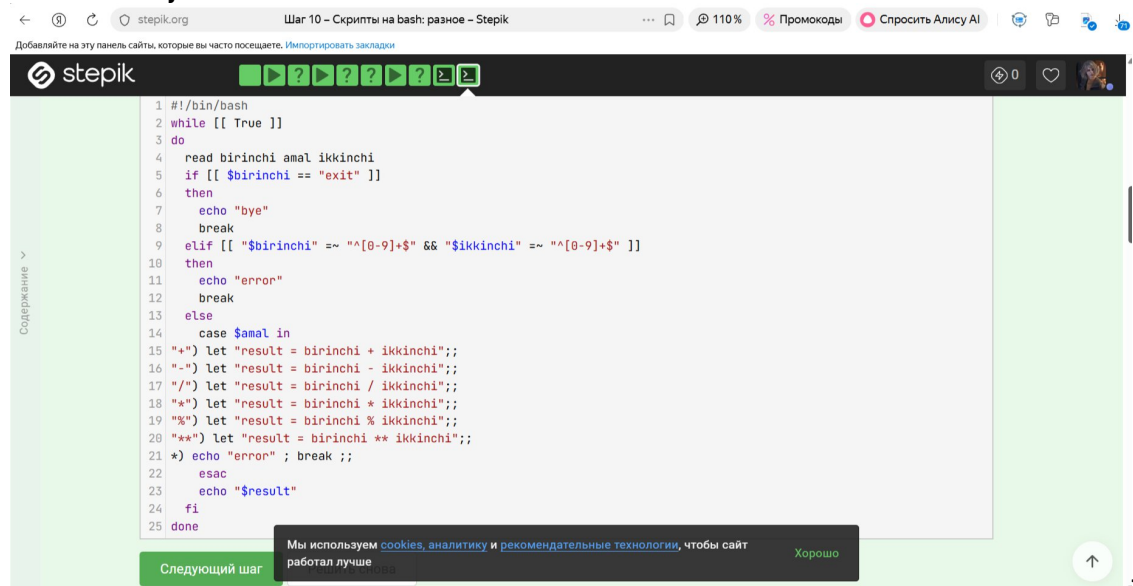
{fig: 040

{fig: 041

¹ 0-9

² 0-9

width=70%}

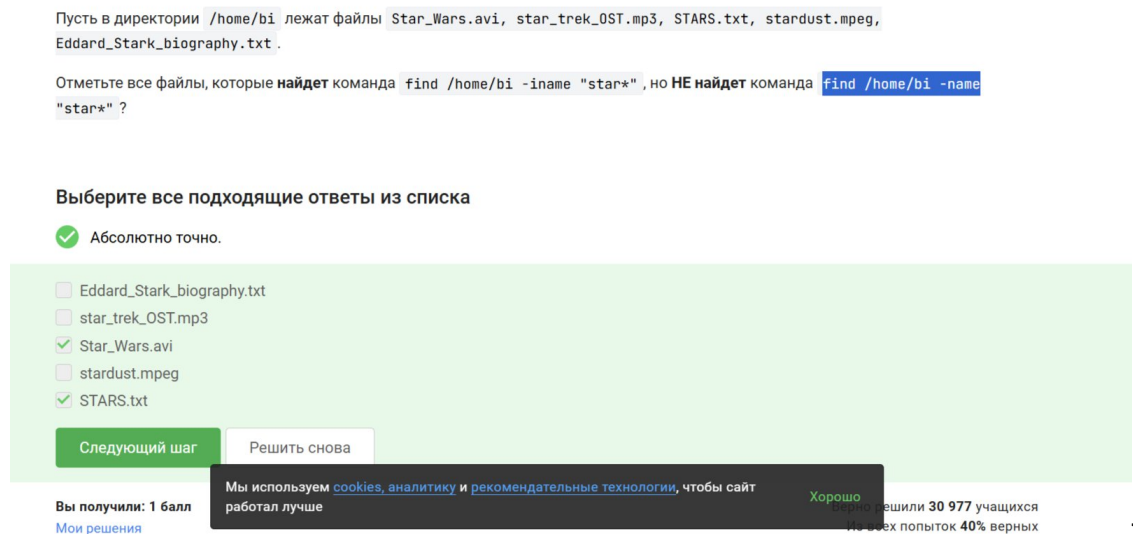


{fig: 042

width=70%}

8. Выполнение подраздела 3.5 Продвинутый поиск и редактирование

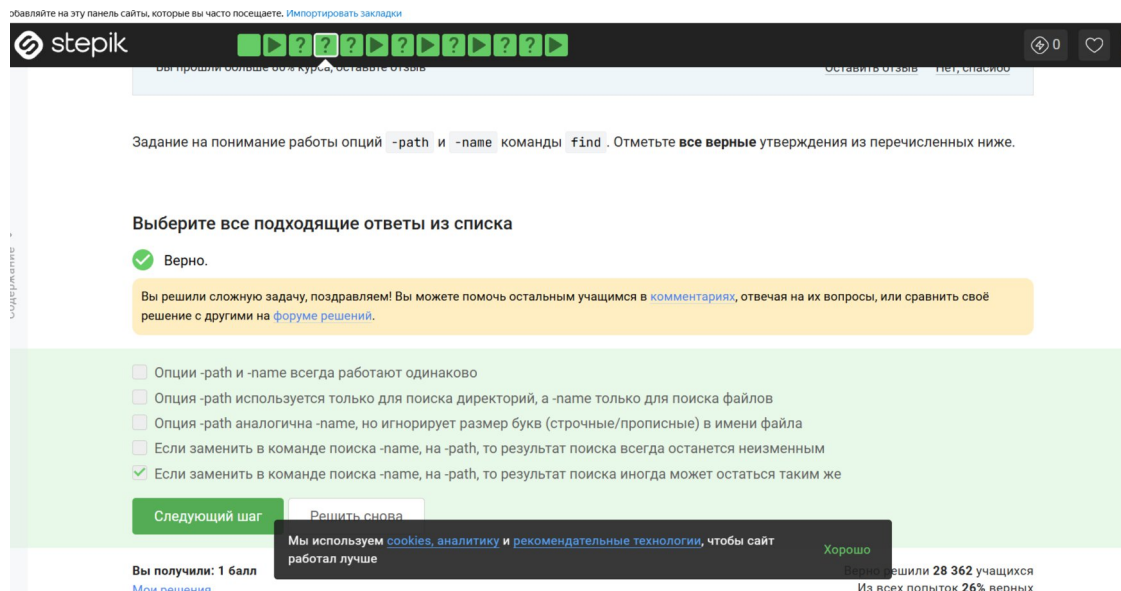
Star_Wars.avi и STARS.txt подходят. Eddard_Stark_biography.txt не найдёт команда `find /home/bi -iname "star*"`. star_trek_OST.mp3, stardust.mpeg - найдёт команда `find /home/bi -name "star*"` ?



{fig: 043

width=70%}

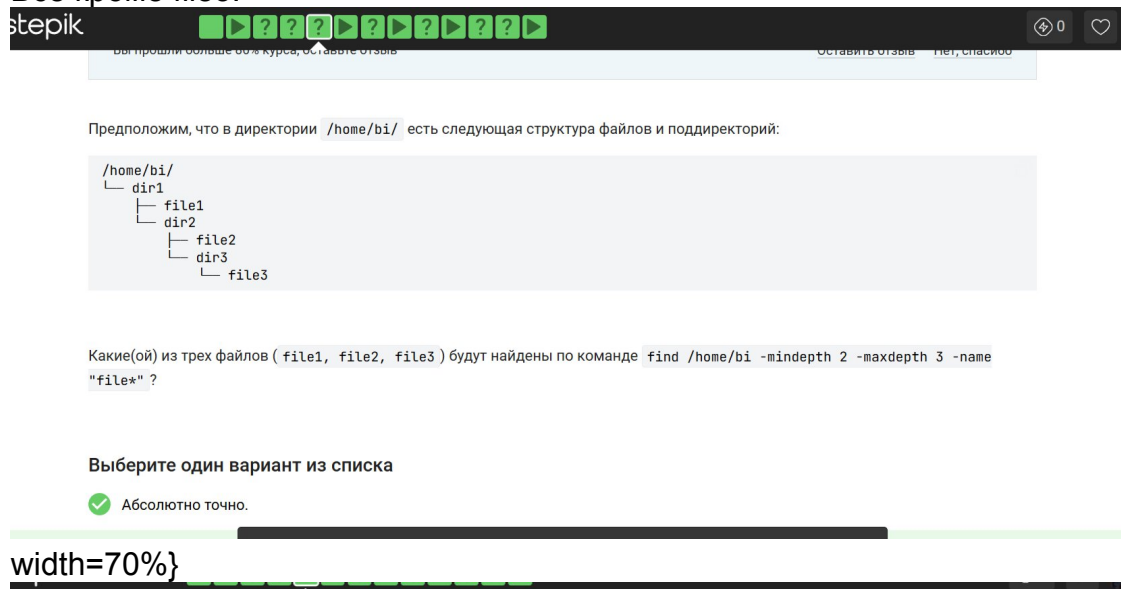
Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же - единственное верное решение. Если файл лежит прямо в текущей директории и его имя совпадает с относительным путём, то результат может совпасть. Частный случай.



{fig: 044}

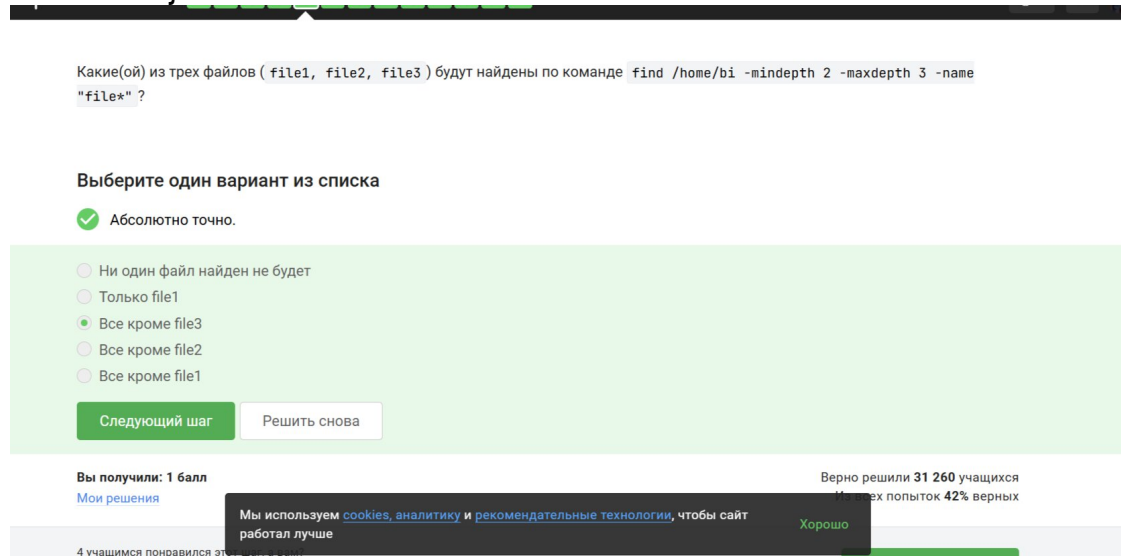
width=70%}

Все кроме file3.



{fig: 045}

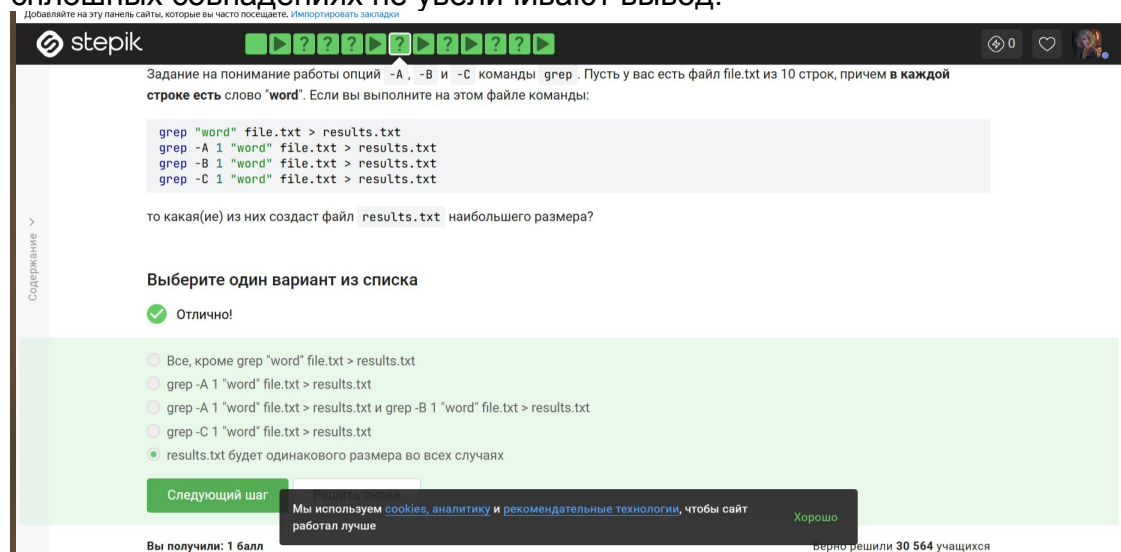
width=70%}



{fig: 046}

width=70%}

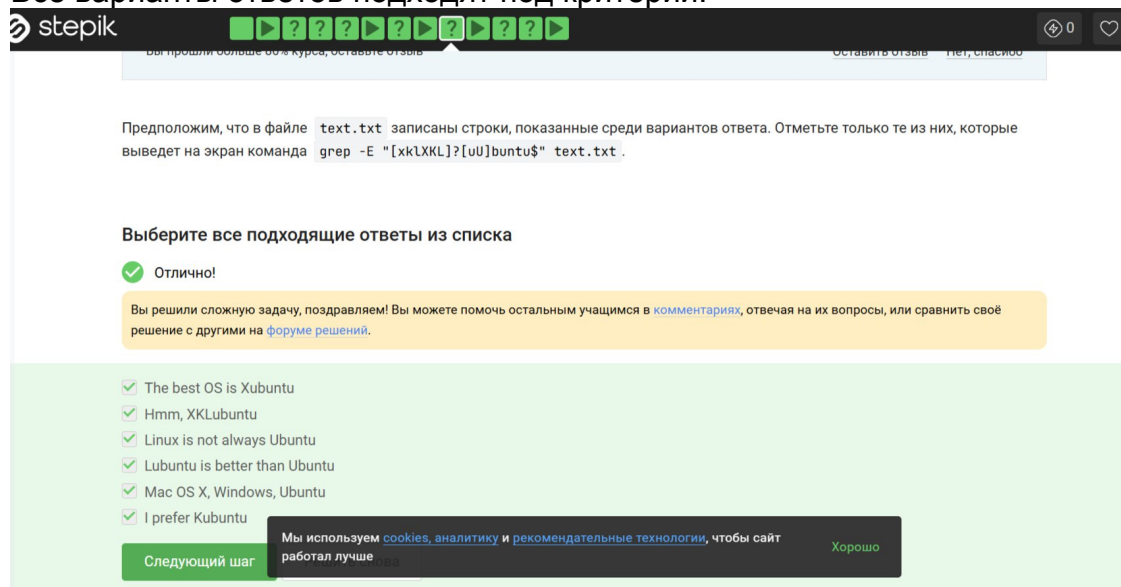
results.txt будет одинакового размера во всех случаях. Все команды выведут весь файл полностью, потому что word есть в каждой строке, а опции -A, -B, -C при сплошных совпадениях не увеличивают вывод.



{fig: 047

width=70%}

Все варианты ответов подходят под критерии.



{fig: 048

width=70%}

Каждая строчка будет выведена два раза. Без -n sed по умолчанию выводит каждую строку автоматически. "[a-z]/p" - команда -p явно печатает строку еще раз, если она соответствует выражению. [a-z] означает 0 или более строчных букв - а это подходит для любой строки.

Что произойдет, если в команде `sed -n "[a-z]*p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка

☒ Отличное решение!

- ☒ Каждая строка будет выведена два раза
- ☐ На экран ничего не напечатается
- ☐ На экран будет выведено всё содержимое файла `text.txt`
- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита

Следующий шаг

Решить снова

Вы получили: 1 балл

[Мои решения](#)

Верно решили 29 965 учащихся

Из всех попыток 40% верных

3 учащимся понравился этот вопрос



Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо

Следующий шаг >

{fig: 049}

width=70%}

`sed 's/[A-Z]{2,} /abbreviation /g' input.txt > edited.txt`. `[A-Z]{2,}` - ищет последовательность из двух и более заглавных латинских букв. Пробел после `{2,}` требует, чтобы после этой последовательности был пробел. `abbreviation` - заменяет найденное на `abbreviation`. `g` - заменяет все вхождения в строке. `input.txt > edited.txt` - читает из `input.txt`, результат пишет в `edited.txt`.

Запишите в форму ниже инструкцию `sed`, которая заменит все "аббревиатуры" в файле `input.txt` на слово "abbreviation" и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что в инструкции должны быть указаны и сам `sed`, и оба файла!

Под "аббревиатурой" будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте не может быть две "аббревиатуры" подряд. Например, текст " YOU YOU and YOU!" является некорректным (в нем есть две "аббревиатуры", но они идут подряд) и на таких примерах мы проверять вашу инструкцию не будем.

Пример: если у вас был текст "Hi, I heard these songs by ABBA, TLA and DM !", то он должен быть преобразован в "Hi, I heard these songs by ABBA, abbreviation and abbreviation !".

Примечание: после вашей замены "аббревиатуры" на слово "abbreviation" количество пробелов в тексте не должно меняться!

Внимание! Во время проверки мы не запускаем команду, которую вы ввели на реальном файле с "аббревиатурами" (это небезопасно, можно же ввести что-то, что не предусмотрено). Значит, что на экран подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а именно `sed` и сделав затем запускаем её смысловую часть (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах.

{fig: 050}

width=70%}

← stepik.org Шаг 12 – Продвинутый поиск и редактирование – Stepik

Добавляйте на эту панель сайты, которые вы часто посещаете. [Импортировать закладки](#)

stepik

Примечание: после вашей замены "аббревиатуры" на слово "abbreviation" количество пробелов в тексте не должно меняться!

Внимание! Во время проверки мы не запускаем команду, которую вы ввели на реальном файле с "аббревиатурами" (это небезопасно, можно же ввести `rm -rf /*`)! Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем запускаем её смысловую часть (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах. К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная "несовместимость" заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. `*`, `+`, `?` и `{}`). Однако эту "несовместимость" легко исправить указав при помощи "(" и ")" какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве "a") нужно записать как `(a)+?` (при этом запись `(a)+?`, конечно же, не поможет).

Напишите текст

✓ Хорошая работа.

`sed 's/[A-Z]\{2,\} /abbreviation /g' input.txt > edited.txt`

Следующий шаг Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше Хорошо

{fig: 051

width=70%}

stepik

Вы прошли больше 60% курса, оставьте отзыв

Оставить отзыв Нет, спасибо

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

✓ Всё правильно.

☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!

☐ `-s, --show-plots-after-exit`

☐ Такой опции не существует

☒ `-p, --persist`

Следующий шаг Решить снова

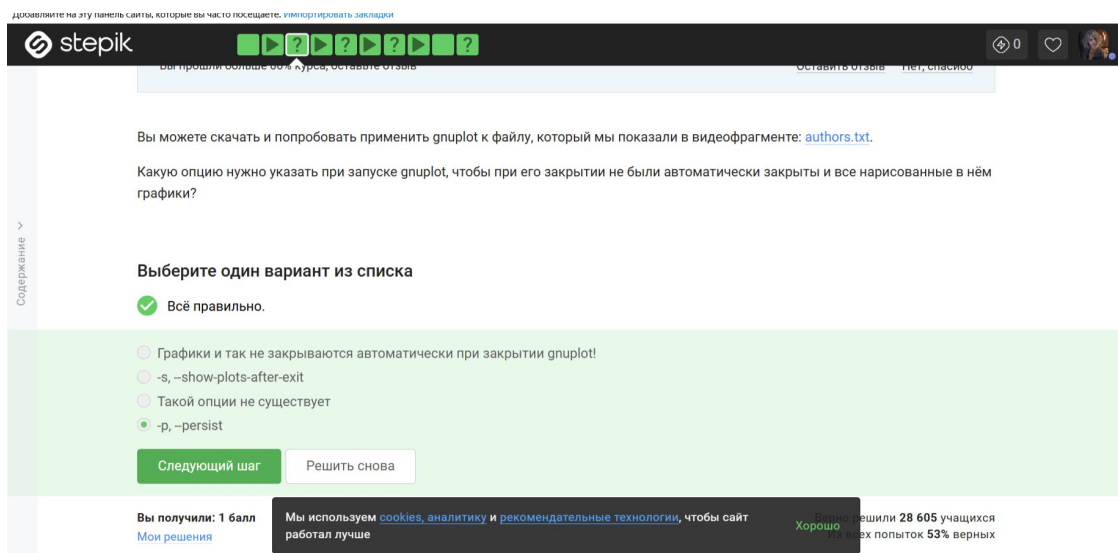
Вы получили: 1 балл Мои решения Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше Хорошо решили 28 605 учащихся всех попыток 53% верных

{fig: 052

width=70%}

9. Выполнение подраздела 3.6 Строим графики в `gnuplot`

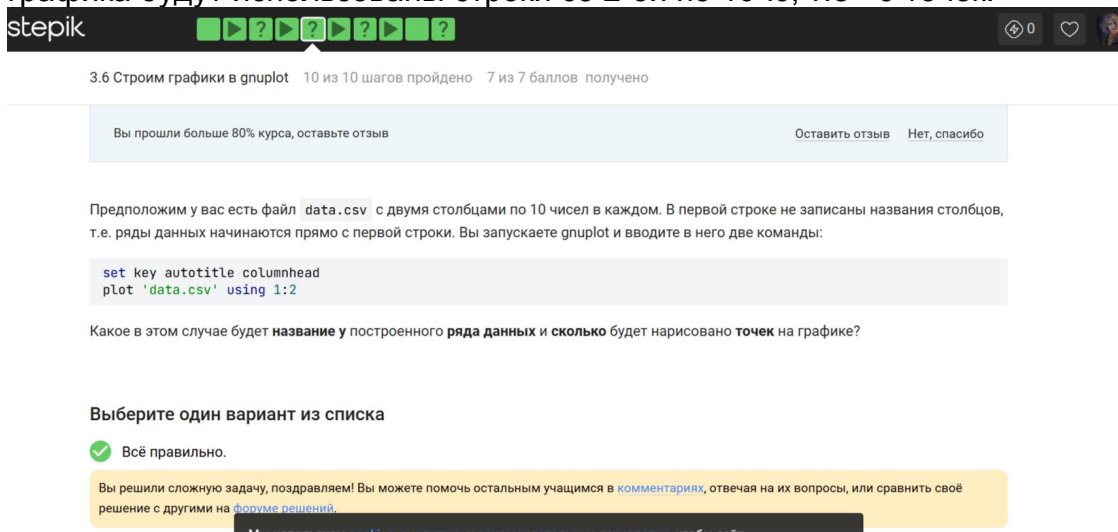
`-p, --persist` заставляет `gnuplot` не закрывать окна с графиками после завершения работы программы.



{fig: 053

width=70%}

Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена). Команда `set key autotitle columnhead` заставляет gnuplot использовать первую строку данных как заголовок, но только для того столбца, по которому строится график. Однако, поскольку в файле нет отдельной строки с заголовками, первая строка будет заголовком, а для построения графика будут использованы строки со 2-ой по 10-ю, т.е. 9 точек.



{fig: 054

width=70%}

stepik

Выберите один вариант из списка

✓ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Название – первое значение из первого столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек
- ☐ Название "popate", нарисовано 10 точек

Следующий шаг Решить снова

Вы получили: 1 балл
Мои решения

Верно решили 27 562 учащихся
Из всех попыток 34% верных

3 учащимся понравился этот шаг

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо Следующий шаг >

{fig: 055}

width=70%}

set xtics - задает пользовательские деления на оси x. В скобках перечисляются пары “подпись” координата.

epik

Вы прошли больше 60% курса, оставьте отзыв

Оставить отзыв Нет, спасибо

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предположим, что вы пишете gnuplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде **"point <номер точки>, value <значение соответствующей переменной>".**

Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20.

Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно!**

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин **конкатенация**, который важен для выполнения данного задания. Под **конкатенацией** обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо

{fig: 056}

width=70%}

epik

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно!**

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин **конкатенация**, который важен для выполнения данного задания. Под **конкатенацией** обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст

✓ Так точно!

set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)

{fig: 057}

width=70%}

$a = a + 1$ - счетчик кадров. $zrot = (zrot + 350) \% 360$ - угол вращения вокруг оси Z увеличивается на 350 градусов, вращение в одну сторону. `set view xrot, zrot` - установка углов обзора. `splot -x**2-y**2` - рисуется параболоид, открытый вниз (вершина в 0, ветви ниже) `pause 0.1` - пауза 0.1 секунды между кадрами. `if(a<50) reread` - повторить 50 кадров.

stepik

Вы прошли больше 60% курса, оставьте отзыв

Оставить отзыв Нет, спасибо

Содержание

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять** и **удалять** инструкции **нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу gnuplot и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в gnuplot работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт

{fig: 058

width=70%}

Напишите текст

✓ Абсолютно точно.

```
a = a+1
zrot=(zrot+350)%360
set view xrot, zrot
splot -x**2-y**2
pause 0.1
if(a<50) reread
```

Следующий шаг Решить снова

Вы получили: 3 балла
Мои решения

Верно решили 21 313 учащихся
Из всех попыток 55% верных

4 учащимся понравился этот шаг, а вам?

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт

Следующий шаг >

{fig: 059

width=70%}

10. Выполнение подраздела 3.7 Разное

Какая команда(ы) установят файлу `file.txt` права доступа `rw-rw-r--`, если изначально у него были права `r--r--r--`. Укажите **все верные** варианты ответа!

Примечание: запись вида команда1; команда2; команда3 означает, что в терминале последовательно выполнились все три команды (сначала команда1, затем команда2 и, наконец, команда3).

Выберите все подходящие ответы из списка

✔ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ chmod 764 file.txt
- ☐ chmod 777 file.txt
- ☐ chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt
- ☐ chmod rwxrw-r-- file.txt
- ☐ chmod u-wx file.txt; chmod g-w file.txt
- ☒ chmod ug+w file.txt

{fig: 060

width=70%} chmod 764 file.txt и chmod ug+w file.txt; chmod u+x file.txt подходят под критерии, остальные - нет.

Верные: `sudo chmod a+w dir`, `sudo chown user:group dir`, `sudo chmod o+w dir`.





Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rw-r--r--` (владелец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды `user` из группы `group` всё-таки сможет создать файл внутри `dir`? Укажите **все верные** варианты ответов!

Примечание: считаем, что все команды выполняются от имени `user`, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно.

Дело в том, что если создать при помощи `sudo` файл с правами `rw-r--r--` в директории, которая принадлежит пользователю, то возникнет любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено удалять **все** файлы внутри его директории) и может прочитать его содержимое (т.к. право `r` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `w` у файла есть только для `root`). При этом некоторые "умные" редакторы, например, `vim` позволят даже редактировать этот файл, но сделают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалять мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `rw-r--r--`, пользователь может сделать с этим файлом почти всё что угодно!

В случае же, когда речь идет о директории созданной `root`, ситуация будет проще: пользователь сможет смотреть её содержимое (у него есть право `r`), но удалять и создавать файлы в ней не сможет (права `w` у него нет).

Важно отметить, что директории в `Linux` это в каком-то смысле *файлы*. Содержимое такого "файла" – это записи о файлах и поддиректориях этой директории.

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше. Вы можете отключить cookies, но это ухудшит работу сайта. Хорошо

просматривать "записи" о файлах и поддиректориях этой директории в ней.

т.е. удалять/создавать файлы/директории в ней.

На самом деле и это еще не всё. Существует так называемый `sticky bit` (атрибут файла или директории), выставление которого

{fig: 061

```
width=70%}
```

stepik

т.е. удалять/создавать файлы/поддиректории в ней.
На самом деле и это еще не всё. Существует так называемый **sticky bit** (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса **Alexey Antipovsky** за помощь в оформлении **Примечания 2!**

Выберите все подходящие ответы из списка

✓ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ✓ sudo chmod a+w dir
- ✓ sudo chown user:group dir
- ✓ sudo chmod o+w dir
- ☐ sudo chmod g+w dir
- ✓ sudo chown user d
- ☐ sudo chown :group

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо

{fig: 062

width=70%}

Верные: Размер файла в байтах, Количество слов, Количество символов.

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

✓ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ✓ Размер файла в байтах
- ✓ Количество слов
- ☐ Количество предложений
- ✓ Количество символов
- ☐ Количество определенных букв (например, количество букв "A")

Следующий шаг

Решить снова

Мы получили: 1 балл

Мои решения

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо

Вы решили 26 076 учащимся

Из всех попыток 23% верных

{fig: 063

width=70%}

Ответ: du -h -s

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо 2048 байт надо вывести 2.0K) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4M (также на экране может быть выведен еще и символ ".", обозначающий, что это размер именно *текущей* директории).

Напишите текст

✓ Правильно, молодец!

du -h -s

Следующий шаг

Мы используем cookies, аналитику и рекомендательные технологии, чтобы сайт работал лучше

Хорошо

{fig:

064width=70%}

Ответ: mkdir dir{1..3}

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✔ Всё получилось!

mkdir dir{1..3}

Следующий шаг

Решить снова

Вы получили: 2 балла
[Мои решения](#)

Мы используем [cookies](#), [аналитику](#) и [рекомендательные технологии](#), чтобы сайт работал лучше

Хорошо решили 25 550 учащихся
Из всех попыток 46% верных

width=70%}

{fig: 065